

FactoryOS LLD

Low Level Architecture - Module Specifications

Document	LLD v1.0 - Low Level Architecture
Scope	8 architectural layers, 13 core modules
Generated	July 21, 2026
Purpose	Detailed module design & integration guide

Architectural Layers

The system is organized into 8 distinct layers, each with well-defined responsibilities and clear integration boundaries. Data flows top-down from user interface through processing layers to response output. Middleware layers (Authentication, Rate Limiter, Logging) operate across all requests.

Module Specifications

Dashboard — LAYER 1: Presentation

Responsibility: User-facing interface for query input, result visualization, and interaction management.

Inputs	Formatted responses, Status updates
Outputs	User queries, Session management
Tech Stack	React/Vue.js, TypeScript, WebSocket for real-time updates
Depends On	Axios API Client

Axios API Client — LAYER 2: Communication

Responsibility: HTTP client for dashboard-to-backend communication. Handles authentication headers, request/response interceptors, and error handling.

Inputs	Response data, Error messages
Outputs	Authenticated HTTP requests
Tech Stack	axios (npm), request timeout handling, retry logic
Depends On	FastAPI Server

FastAPI Server — LAYER 2: Communication

Responsibility: REST/async HTTP server. Routes requests to appropriate handlers, validates input, and orchestrates downstream processing.

Inputs	HTTP requests, Authentication context
Outputs	Parsed requests to handlers
Tech Stack	FastAPI, Uvicorn ASGI server, Pydantic for validation
Depends On	Authentication

Authentication — LAYER 3: Middleware

Responsibility: Token validation (JWT/OAuth), user identity verification, and permission checks. Runs before any downstream processing.

Inputs	HTTP headers with token
Outputs	Verified user context, Access token
Tech Stack	PyJWT, OAuth2, API keys, or session tokens
Depends On	FastAPI Server

Rate Limiter — LAYER 3: Middleware

Responsibility: Request throttling to prevent abuse. Tracks per-user or per-IP request counts and enforces quotas.

Inputs	Request metadata (IP, user)
Outputs	Pass/reject decision
Tech Stack	Redis-backed counters or in-memory sliding window
Depends On	FastAPI Server

Logging — LAYER 3: Middleware

Responsibility: Structured logging of all requests, responses, errors, and system events. Includes audit trail for compliance.

Inputs	Request/response data, Error messages
Outputs	Log entries to persistent storage
Tech Stack	Python logging (stdlib), ELK Stack, or Cloud Logging
Depends On	FastAPI Server

Agent Orchestrator — LAYER 4: Orchestration

Responsibility: Central coordinator. Routes authenticated requests to specialized agents, manages agent lifecycle, handles concurrency, and collects results.

Inputs	Authenticated request, Agent results
Outputs	Agent tasks, Result aggregation
Tech Stack	LangGraph, asyncio, task queues (Celery/RQ)
Depends On	All Agent modules

Industrial GPT Agent — LAYER 5: Agents

Responsibility: Primary reasoning agent. Understands domain-specific industrial queries, coordinating with other agents to synthesize a response.

Inputs	User query, Context from other agents
Outputs	Reasoning chain, Sub-task delegations
Tech Stack	Claude API, LangChain, function calling
Depends On	Knowledge Agent

Maintenance Agent — LAYER 5: Agents

Responsibility: Specialized for predictive maintenance, anomaly detection, and asset health scoring. Returns maintenance recommendations.

Inputs	Asset telemetry, Historical maintenance data
Outputs	Maintenance scores, Risk assessments
Tech Stack	scikit-learn, XGBoost, time-series analysis
Depends On	Knowledge Agent

Compliance Agent — LAYER 5: Agents

Responsibility: Validates responses and actions against regulatory and compliance rules. Adds policy disclaimers and audit metadata.

Inputs	Proposed response, Regulatory framework
Outputs	Compliance flags, Audit records
Tech Stack	Rules engine (Drools) or custom Python validators
Depends On	Knowledge Agent

Knowledge Agent — LAYER 5: Agents

Responsibility: Manages knowledge base access. Coordinates RAG and GraphRAG queries, filters results, and synthesizes evidence.

Inputs	Query context, Retrieval requests
Outputs	Ranked evidence, Source citations
Tech Stack	LangChain, vector DB integration, ranking algorithms
Depends On	GraphRAG

Drawing Agent — LAYER 5: Agents

Responsibility: Generates technical diagrams, flowcharts, and visualizations based on query context. Returns SVG/PNG artifacts.

Inputs	Query intent, Data to visualize
Outputs	SVG diagrams, Visual explanations
Tech Stack	Graphviz, ReportLab, or D3.js server-side rendering
Depends On	None

GraphRAG — LAYER 6: Knowledge

Responsibility: Structured knowledge graph queries. Retrieves relationships, entity properties, and hierarchical knowledge using graph traversal.

Inputs	Structured query, Entity names
Outputs	Graph query results, Relationship chains
Tech Stack	Neo4j, Apache TinkerPop, or custom graph DB
Depends On	None

Retriever — LAYER 6: Knowledge

Responsibility: Fetches documents from vector index using semantic similarity. Handles embedding lookups and document ranking by relevance score.

Inputs	Query embedding
Outputs	Top-K documents, Relevance scores
Tech Stack	FAISS, Pinecone, Weaviate, or Milvus
Depends On	None

Ranking — LAYER 6: Knowledge

Responsibility: Re-ranks retriever results by relevance, date, source authority, and confidence. Applies diversity and deduplication filters.

Inputs	Retriever results, Query context
Outputs	Ranked document list
Tech Stack	Learning-to-rank models, BM25, or rule-based scoring
Depends On	Retriever

Prompt Builder — LAYER 7: Processing

Responsibility: Constructs the final prompt by merging retrieved documents, agent reasoning chains, and user context. Manages token budget and relevance.

Inputs	Query, Retrieved evidence, Agent outputs
Outputs	Final prompt text, Token metadata
Tech Stack	Template engines (Jinja2), prompt optimization
Depends On	Knowledge Agent

LLM — LAYER 7: Processing

Responsibility: Invokes the large language model (Claude or alternative). Streams tokens, manages context windows, and handles API errors/retries.

Inputs	Prompt, System instructions
Outputs	Generated text, Token usage metrics
Tech Stack	Claude API, OpenAI SDK, or local LLM (Ollama)
Depends On	None

Formatter — LAYER 7: Processing

Responsibility: Parses raw LLM output into structured JSON. Extracts claims, citations, confidence scores, and formatting directives.

Inputs	Raw LLM output
Outputs	Structured response object
Tech Stack	JSON parsing, regex, or pydantic models for validation
Depends On	LLM

Response — LAYER 8: Output

Responsibility: Final output layer. Adds response metadata (timestamp, request_id), applies final transformations, and streams to client.

Inputs	Structured response from Formatter
Outputs	Formatted JSON response
Tech Stack	FastAPI response models, streaming JSON
Depends On	Formatter

Integration & Data Flow

Request Path (Top-Down):

Dashboard → Axios Client → FastAPI Server → Authentication → Rate Limiter → Logging → Agent Orchestrator

Agent Processing Path:

Agent Orchestrator routes to specialized agents (Industrial GPT, Maintenance, Compliance, Knowledge, Drawing). Each agent operates concurrently:

- Knowledge Agent queries GraphRAG and Retriever in parallel
- Ranking module re-scores results
- Agents feed context to Prompt Builder

Response Path (Bottom-Up):

Prompt Builder → LLM → Formatter → Response → FastAPI → Axios Client → Dashboard

Cross-Cutting Concerns:

- Logging happens at every stage (all layers)
- Authentication is verified once at entry; context is passed downstream
- Rate limits are checked before agent execution
- Error handling is standardized across all layers (retry logic, circuit breakers)

Deployment & Scalability

Stateless Design: All modules are stateless (except persistent storage layers like GraphRAG and Retriever). This enables horizontal scaling — add more FastAPI instances, agent workers, or LLM replicas as load increases.

Concurrency Model: Use async/await throughout for I/O-bound operations (API calls, database queries). CPU-bound agents (XGBoost models, ranking algorithms) run in thread pools or separate processes.

Caching Strategy:

- Query cache: Identical queries return cached responses (with TTL)
- Embedding cache: Pre-computed embeddings for common terms
- Agent result cache: Consensus results from multiple agents cached by context hash

Monitoring & Observability:

- Latency tracking per module
- Error rates and retry success metrics
- Token usage tracking for LLM cost management
- Agent consensus scores (when multiple agents address the same query)

Implementation Priority

Phase 1 (MVP): FastAPI, Dashboard, Agent Orchestrator, Industrial GPT Agent, Knowledge Agent, basic RAG (Retriever only), LLM, Formatter.

Phase 2: GraphRAG, Ranking, Maintenance & Compliance agents, Drawing Agent.

Phase 3: Advanced logging, monitoring dashboards, caching layers, performance optimization.

FactoryOS LLD v1.0 | Every block becomes one module